

PriceRight: An Accountability Layer for x402 Agent Commerce on Base

Dipankar Sarkar

2026

Abstract

Autonomous agents are starting to transact with each other on Base, where x402 makes an agent-to-agent USDC payment a single HTTP round trip, but an agent that is paid to produce a judgement has no skin in the game: if it answers wrong, it keeps its fee and moves on. PriceRight closes that gap by making the right to answer a paid resource and the answer a collateralised commitment. The claim endpoint replies 402 Payment Required with x402 payment requirements; the resolver signs an EIP-3009 TransferWithAuthorization and presents it in an X-PAYMENT header, and the arena settles that authorisation on-chain, so the fee moves by signature rather than by a prior allowance and is bound to one task by a nonce that commits to the arena address and the chain id. In the same call the resolver’s collateral is custodied per task in a vault keyed by its ERC-8004 identity, which is a real ERC-721. Settlement is a pure function of the committed verdict and a truth the poster hash-committed in advance: correct returns the fee, the bounty and the collateral and writes a score of 100 into the ERC-8004 reputation registry; wrong moves the collateral to the poster and writes a zero. Commit and reveal deadlines with permissionless timeouts mean neither party can freeze the escrow, and no terminal path leaves value stranded. We report 46 Solidity tests, 53 Python tests, a cross-language pin in which a signature produced by this paper’s pure-Python secp256k1 is verified by Solidity’s ecrecover while the Python suite regenerates the same vector from the Solidity constants, and a recorded local devnet run whose transaction hashes are reproduced in the interface.

1 Problem

Agent-to-agent commerce assumes an agent that takes payment for work will do the work well, but a fee alone creates no accountability. If a resolver agent is paid to answer a subjective yes/no question (“did this event happen?”, “did this project ship?”) and answers wrong, a naive market simply pays it and forgets. The result is an economy where confidently wrong agents are indistinguishable, ex ante, from careful ones, and where

a buyer of judgements cannot price the risk of a bad answer.

Two ingredients are missing. First, a payment rail that gates access to the task so that answering is never free [4, 7]. Second, a portable, on-chain identity that carries reputation and bonded value, so a wrong answer costs the agent something it cannot quietly reset [9]. Proof-of-stake systems have long used bonded stake plus slashing to make misbehaviour expensive [1, 8]; PriceRight applies the same economic logic to the outputs of autonomous agents.

2 Related Work

ERC-8004 Trustless Agents [9] specifies an Identity registry (an agent as an ERC-721 [6] whose token id is the agent id), a Reputation registry in which feedback is permissionless and readers filter by client, and a Validation registry for third-party attestation. PriceRight implements all three and adds the economics the standard deliberately omits: a collateral vault keyed by agent id, so a score of zero is accompanied by value that actually moved. Because reputation is keyed by agent id rather than by address, a transferred identity carries its record with it. The fee token follows ERC-20 [10] and additionally implements EIP-3009 transfer authorisations, as USDC does.

x402 [4] is an HTTP-native payment standard built on the 402 Payment Required status code [7]: a resource answers 402 with an accepts array of payment requirements, the client returns a payment payload in an X-PAYMENT header, and a facilitator verifies and settles it. On EVM chains the exact scheme settles by EIP-3009 over USDC on Base, which is what PriceRight implements: the right to claim and be scored on a task is the paid resource, and the arena is the party that submits the payer’s signed authorisation.

Bonded stake and slashing are the core accountability mechanism of proof-of-stake consensus [1], where a validator that signs conflicting messages loses stake. Adversarial value-extraction on public chains [5] motivates a deterministic, commit-based settlement so the outcome cannot be steered after the fact. Content integrity

uses keccak-256 [2], the hash Ethereum exposes as keccak256 [3].

3 Design

PriceRight is six contracts and a resolver agent.

ERC-8004 registries. IdentityRegistry is an ERC-721 whose token id is the agent id; supportsInterface reports 0x80ac58cd and 0x5b5e139f, and identities transfer and approve like any other token. ReputationRegistry exposes giveFeedback to anyone and getSummary(agentId, clients, tag) to readers, who decide whose feedback counts; the arena writes as an ordinary client, not as an administrator. ValidationRegistry records a request and, later, a response.

AgentStakeVault. ERC-8004 says nothing about money, so the economics live in an extension. Collateral is real ERC-20 held by the vault under a key the arena derives from (arena, taskId) and tagged with the agent id it backs. Every bond ends in exactly one of two terminal states, and both move tokens: release to a named address, or slash to a named beneficiary. There is no path that leaves collateral in the vault, which the suite asserts by draining the system after each settlement.

AgentArena. A poster calls postTask with a bounty and a salted commitment $c = \text{keccak256}(\text{uint8}(t) \parallel s)$, fixing the truth t before any resolver can commit. A resolver calls claimTask with an x402 exact payment: the EIP-3009 fields plus (v, r, s) . The arena requires that the payer is the caller, that the value equals the posted fee, and that the nonce equals

$\text{keccak256}(\text{scope} \parallel \text{arena} \parallel \text{chainId} \parallel \text{taskId} \parallel \text{agentId}),$

then calls transferWithAuthorization and checks the balance actually moved. That nonce is the binding: an authorisation signed for one task cannot be replayed against another task, another arena or another chain, and the token's own nonce map makes it single-use. Collateral is bonded in the same call. The resolver then calls commitVerdict, which stores keccak256 of its reasoning and files a validation request whose request hash is derived from it, before the truth is revealed, so the attestation cannot be back-dated. Finally the poster calls settle, revealing (t, s) ; the contract checks the reveal reproduces c and compares:

correct $\Rightarrow$ fee+bounty+collateral to the resolver, feedback 100
 wrong $\Rightarrow$ fee+bounty+collateral to the poster, feedback 0.

Liveness. A claim sets a commit deadline and a commit sets a reveal deadline. After either passes, anyone may call the corresponding timeout. A resolver that never commits forfeits its fee to the poster but keeps its collateral, since it gave no wrong verdict; a poster that never reveals pays the resolver the fee, the bounty and its collateral back, which makes withholding a reveal strictly worse than revealing whatever the truth turns out to be. Payouts are credits withdrawn by pull, so one unlucky transfer cannot brick a settlement.

Two backends, one interface. The agent talks to a chain object. JsonRpcChain builds, RLP-encodes, signs and broadcasts EIP-1559 transactions and reads state with eth_call. InMemoryChain executes the same state machine in process, including EIP-3009 verification against a real EIP-712 digest with real signature recovery, so a forged or replayed authorisation fails there exactly as it fails in the token. Configuration selects between them in one place, and a half-configured environment raises rather than quietly running the mirror.

The resolver does not know the answer. The poster holds (t, s) ; the resolver is handed a claim and an evidence bundle. EvidencePolicy parses a threshold claim such as ETH/USD ≥ 4000 @ block 21451200, keeps the readings that match both the symbol and the block, takes their median and compares. When the evidence does not cover the claim it returns an abstention and the agent declines to pay at all. The contrasting agent, StaleCachePolicy, evaluates the same comparison against a week-old cached quote. It is not adversarial and it is not rigged to lose: given a threshold its cache clears, it wins.

4 The Hero

Two agents, one arena, one rule, on a live node. Both pay the same x402 fee. The first reads three oracle readings, commits Yes, and at settlement is credited the fee, the bounty and its collateral, with a score of 100 written to the reputation registry. The second answers from its cache, commits No, and at settlement its collateral is moved to the poster and a zero is written against its identity. The difference between them is diligence, not honesty, which is the failure mode a market of paid agents actually has.

What makes it a hero rather than an animation is that a reviewer can leave the room and check it: cast logs shows the Slashed event, cast call on the vault returns the slashed amount for that agent id, and the interface displays the transaction hashes from the same recorded

5 Evaluation

The contracts compile under solc 0.8.24 with no vendored dependencies and pass 46 Foundry tests: the lifecycle, both settlement paths, the value-conservation invariant after each, both permissionless timeouts, the deadline guards, the x402 binding and replay properties, and ERC-721 and reputation conformance. The Python side passes 53 tests covering the same state machine, the policy’s dependence on evidence, the abstention path, the streamed run events and every rejection reason the facilitator can return.

Two results are worth separating from the counts. First, the fee genuinely moves by signature: the resolver holds no allowance for the arena, so `test_fee_moves_by_signature_not_by_allowance` would fail if the payment were decorative. Second, the two languages are pinned to each other. The Python client signs with the pure-Python `secp256k1` in this repository; a vector it produced is fed to Solidity, which re-derives the EIP-712 digest and recovers the payer with `ecrecover`, while the Python suite reads those constants back out of the Solidity file and regenerates them. Editing either side to fit the other fails one of the two tests.

```
$ forge test
46 passed; 0 failed
$ pytest tests -q
53 passed
$ ./scripts/devnet.sh
agent #1 evidence -> CORRECT +110 tUSD, collateral
    returned, 100%
agent #2 stale-cache -> WRONG 50 tUSD collateral seized,
    0%
$ cast call $VAULT 'slashedOf(uint256)(uint256)' 2
50
```

6 Limitations

The system has not yet been deployed to Base Sepolia: no funded testnet key was available in this environment, so there is no explorer link, and what has actually been executed is the local devnet run described above; the RPC adapter is network-agnostic and the same code path runs against Base Sepolia once a funded key is supplied, which is the milestone-1 target. No hosted CDP facilitator has been called either, because that needs credentials: the remote rail is exercised against a conformant facilitator served over HTTP by the test suite, which covers the approved, refused and unreachable cases. Verification can be remote but settlement cannot, because `claimTask` consumes the authorisation itself; a facilitator that broadcast it first would burn the nonce and make the claim impossible.

Ground truth is still supplied by the poster under a commitment. The commitment binds the poster to one

value in advance and the reveal timeout removes the incentive to withhold, but neither proves the value is real; sourcing truth from an oracle or a dispute game is the obvious next step. The collateral is a fixed per-task amount rather than a function of confidence or repeat offences. The resolver is a deterministic evidence parser rather than a model, which is a choice made for reproducibility in front of a reviewer: a model fits behind the same `decide(claim, evidence)` interface, and the arena does not care how a verdict was formed. Finally, the ERC-8004 registries here are a self-contained implementation of the standard’s structure rather than the reference deployment, and they omit optional parts of it; `docs/HONESTY.md` enumerates exactly which.

References

- [1] Iddo Bentov, Rafael Pass, and Elaine Shi. Snow White: Robustly Reconfigurable Consensus and Applications to Provably Secure Proof of Stake. In Financial Cryptography and Data Security (FC). Springer, 2019.
- [2] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. The Keccak Reference, Version 3.0. <https://keccak.team/files/Keccak-reference-3.0.pdf>, 2011.
- [3] Vitalik Buterin. A Next-Generation Smart Contract and Decentralized Application Platform. Ethereum White Paper, 2014. <https://ethereum.org/en/whitepaper/>.
- [4] Coinbase. x402: An Open Standard for Internet-Native Payments. <https://www.x402.org>, 2025. HTTP 402-based agentic payment protocol.
- [5] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability. In 2020 IEEE Symposium on Security and Privacy (SP), pages 910–927, 2020.
- [6] William Entriken, Dieter Shirley, Jacob Evans, and Nastassia Sachs. ERC-721: Non-Fungible Token Standard. Ethereum Improvement Proposals, no. 721, 2018. <https://eips.ethereum.org/EIPS/eip-721>.
- [7] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP Semantics. RFC 9110, Internet Engineering Task Force, 2022. Defines the 402 Payment Required status code.
- [8] Satoshi Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System. 2008. <https://bitcoin.org/bitcoin.pdf>.

- [9] Marco De Rossi, Davide Cripis, and Jordan Ellis. ERC-8004: Trustless Agents. Ethereum Improvement Proposals, no. 8004, 2025. <https://eips.ethereum.org/EIPS/eip-8004>.
- [10] Fabian Vogelsteller and Vitalik Buterin. ERC-20: Token Standard. Ethereum Improvement Proposals, no. 20, 2015. <https://eips.ethereum.org/EIPS/eip-20>.